Week-2

1.Use **ping, tracert, and nslookup** commands.

2.Demo: **Packet capture using Wireshark** to observe HTTP traffic.

1.Use **ping, tracert, and nslookup** commands.

1) `ping` — check reachability & latency

What to look for: packet loss %, round-trip times (min/avg/max).
Class talk point: If ping fails, could be firewall blocking ICMP, host down, or wrong route.

```
ping -c 4 8.8.8.8       # send 4 ICMP echo requests to Google DNS

ping -c 4 example.com   # ping a hostname
```

2) `traceroute` / `tracert` — show route packets take

What to look for: each hop (router) IP and the RTTs to that hop. Useful for diagnosing where packets slow down or are dropped.

```
traceroute example.com
# on some systems you may need sudo: sudo traceroute example.com
```

3) `nslookup` (and `dig`) — DNS queries & troubleshooting

Nslookup (interactive and single-shot):

```
# Quick query
nslookup example.com

# Query a specific DNS server
nslookup example.com 8.8.8.8

# Interactive mode
nslookup
> server 8.8.8.8
> set type=mx
> example.com
> exit
```

dig (more informative) — Linux / macOS / Kali

```
dig example.com          # A record
dig example.com MX +short  # mail exchange records (short output)
dig +trace example.com     # full DNS trace
```

What to look for: resolved IPs, TTLs, MX/NS records. Use `dig +short` to get a quick answe

2.Packet capture with Wireshark to observe HTTP traffic

Goal

Capture **unencrypted HTTP** request/response and show students:

- HTTP request line (method, path)

- Host header
- User-Agent header
- Response status code
- Follow TCP stream to see raw HTTP content

Note: HTTPS (port 443) is encrypted — you will not see request body or headers in plaintext. For demo show HTTP on port 80 (or run a local HTTP server).

1) Setup a local HTTP server (safe & simple)

Run on your Kali / demo machine (or teacher machine). This serves `index.html` from the current directory on port 8000:

```
# in a folder with an index.html
python3 -m http.server 8000 --bind 0.0.0.0
```

Access it from the same machine: `http://127.0.0.1:8000/` or from students' machines using your demo machine IP `http://<teacher-ip>:8000/`.

2) Start Wireshark and begin capture (GUI steps)

1. Open **Wireshark** (requires root privileges on Linux).
2. Select the correct interface:
    - For local HTTP server on same host capture on `lo` (loopback) on Linux/Mac, or `Ethernet/Wi-Fi` if server is on a different machine.
3. Click **Start** to begin capture.

4. Optionally set a capture filter to reduce noise (capture filter — BPF), e.g.:
    - Capture only HTTP: `tcp port 80`
    - Capture only traffic to/from a host: `host 192.168.56.101`
      (Note: capture filters are applied before capture; display filters are applied after.)

3) Generate HTTP traffic (examples)
From a student machine or same host:

```
# Simple GET with curl (shows headers in Wireshark)
curl http://<teacher-ip>:8000/

# Request a specific path
curl http://<teacher-ip>:8000/index.html

# Request with a custom User-Agent
curl -A "StudentDemo/1.0" http://<teacher-ip>:8000/
```

4) Stop capture after a few requests

Click the red **Stop** button in Wireshark. Save the capture: `File → Save As` → `.pcapng`.

5) Use Wireshark display filters to find HTTP traffic

In the Wireshark **Display Filter** bar enter:

- `http` — show HTTP protocol packets

- `http.request` — show only HTTP request packets
- `http.response` — show only HTTP responses
- `http.request.method == "GET"` — show only GET requests
- `http.host == "example.com"` — filter by Host header
- `tcp.port == 8000` — if using nonstandard port

6) Inspect a single HTTP transaction

1. Click a packet that is an **HTTP GET** (the Info column will show `GET /index.html HTTP/1.1`).
2. In the middle pane expand **Hypertext Transfer Protocol** to see:
    - **Request Method** (GET)
    - **Request URI** (/index.html)
    - **Host:** header (Host: `<teacher-ip>:8000`)
    - **User-Agent:** header
3. Click the matching **HTTP 200 OK** response packet. Expand **Hypertext Transfer Protocol** to see:
    - **Response Code** (200)
    - **Content-Type**

- **Content-Length**

4. Right-click on request or response → **Follow** → **TCP Stream** to view the full request/response exchange in plain text.

7) Extra Wireshark goodies (useful demo points)

- **Follow TCP Stream** (shows entire conversation).
- **Export HTTP objects**: `File → Export Objects → HTTP` (saves downloaded files from the capture).
- **Endpoint statistics**: `Statistics → Endpoints` to show IP pairs and traffic volume.
- **HTTP request/response time**: use `Statistics → HTTP` (if available) to show timings.
- **Apply column**: add `http.host` or `http.user_agent` columns for easy inspection.

8) CLI alternative: `tshark` / `tcpdump`

If you want to capture from the command line (teacher demo or scripted):

```
sudo tcpdump -i eth0 tcp port 80 -w http_demo.pcap
# or loopback
sudo tcpdump -i lo tcp port 8000 -w http_demo_lo.pcap
```

View HTTP requests with tshark (live):

```
# Show HTTP request lines as they appear
sudo tshark -i eth0 -Y "http.request" -T fields -e ip.src -e http.host -e http.request.uri -e http.user_age
```

Run on demo host:

```
# 1) start server
mkdir -p ~/phish_demo;
cd ~/phish_demo
echo '<html><body><h1>Training page</h1></body></html>' > index.html
python3 -m http.server 8000 --bind 0.0.0.0 &      # note the & runs in background
SERVER_PID=$!

# 2) start capture (loopback example)
sudo tcpdump -i lo tcp port 8000 -w /tmp/http_demo.pcap &
TCPDUMP_PID=$!

# 3) from student or same host, run curl
curl http://127.0.0.1:8000/index.html

# 4) stop capture and server
sudo kill $TCPDUMP_PID
kill $SERVER_PID

# 5) open the pcap in wireshark
wireshark /tmp/http_demo.pcap &
```

Week-3

A — SQL injection demo with DVWA (Damn Vulnerable Web App)

Goal: show how simple SQLi works in a deliberately vulnerable app, then how to mitigate it.

Safety first

- Run DVWA only in an isolated VM or Docker on your local host (e.g., Kali VM or separate lab VM).
- Do not expose DVWA to the Internet.

1. Safety & prep (READ THIS)

- **Only run DVWA in a lab VM** (Kali VM or a local machine that is not exposed to the Internet).
- Don't try destructive payloads (`DROP`, `DELETE`, etc.) — demo data extraction and authentication bypass only.
- If using other students' machines, get written permission.
- Revert VM snapshots when finished.

2. Quick install & start DVWA on Kali (Docker — recommended)

```
# install docker/docker-compose if not present
sudo apt update
sudo apt install -y docker.io docker-compose

# enable and start docker
sudo systemctl enable --now docker

# make demo dir and compose file
mkdir -p ~/dvwa && cd ~/dvwa
cat > docker-compose.yml <<'YAML'
version: "3"
services:
  dvwa:
    image: vulnerables/web-dvwa:latest
    ports:
      - "80:80"
    environment:
      MYSQL_ROOT_PASSWORD: root
```

```
   restart: unless-stopped
YAML

# start DVWA
docker-compose up -d

# check status
docker ps | grep dvwa
```

Open `http://<kali-ip>/` or `http://localhost/` in the browser. (If you run Kali as a VM use the VM IP.)

3. DVWA first steps (browser)

1. Navigate to DVWA: `http://<kali-ip>/`
2. Login: default `admin` / `password` (if prompted).
3. Go to **Setup / Reset Database** → click **Create / Reset Database**. Wait until success message.
4. Go to **DVWA Security** and set **Low** (for demonstration).

4. Understand the vulnerable page (SQL Injection GET)

- Menu → **SQL Injection (GET)**
- The page will have an input parameter like `id=` in the URL (e.g., `vulnerabilities/sqli/?id=1&Submit=Submit`).

This page directly inserts your input into a SQL query without sanitization, e.g.:

```
SELECT first_name, last_name FROM users WHERE user_id = 'USER_INPUT';
```

5. Demo 1 — Simple non-destructive SQLi (Authentication bypass / always true)

In the input field enter:

```
1' OR '1'='1
```

Then Submit.

What happens: the injected `OR '1'='1'` makes the WHERE condition always true and the app returns all rows or bypasses logic — demonstrates how unchecked input changes query meaning.

Why it works (conceptual):

- Original query: `... WHERE user_id = '1'`
- After input: `... WHERE user_id = '1' OR '1'='1'`
- `'1'='1'` is always true → rows are returned.

Show students: browser URL in devtools/network shows the GET param; you can also show the query in DVWA output if it prints SQL.

6. Demo 2 — Extract database information (safe read-only)

Try a union-based read (be cautious; DVWA supports learning):

```
-1' UNION SELECT null, table_name FROM information_schema.tables --
```

Or simpler, test retrieving usernames:

```
1' UNION SELECT user,password FROM users --
```

Caution: Use only read-only queries. Explain that `UNION SELECT` merges attacker-controlled result rows with the query result.

If DVWA escapes or blocks some forms on Medium/High, note this — you can switch security back to Low to demonstrate.

7. Observe requests (how to show HTTP flow)

- Open **Browser DevTools** → **Network** and submit the form. Show the full request URL (GET) or request body (POST).
- Alternatively run a local packet capture while you submit:

```
# capture HTTP traffic on Kali (if DVWA hosted there)
sudo tcpdump -i lo tcp port 80 -w /tmp/dvwa_http.pcap
# after demo: open in Wireshark
wireshark /tmp/dvwa_http.pcap &
```

8. SQL Injection (POST) demo

- Go to **SQL Injection (POST)** page in DVWA. Enter payload in the POST field:

  ' OR '1'='1

  Submit and show request body using DevTools → Network to demonstrate how POST data is exploited in the backend SQL

9. Optional: Automated test with sqlmap (lab-only)

Use `sqlmap` to show automation of extraction. First capture or craft the vulnerable request. Example quick `sqlmap` against a vulnerable GET parameter:

```
# basic discovery (replace <target> with your DVWA URL)
```

```
sqlmap -u "http://<kali-ip>/vulnerabilities/sqli/?id=1&Submit=Submit" --batch --level=1 --risk=1
```

To dump a table (read-only):

```
# enumerate DBs
sqlmap -u "http://<kali-ip>/vulnerabilities/sqli/?id=1" --dbs --batch
# then choose a DB and dump tables (example)
sqlmap -u "http://<kali-ip>/vulnerabilities/sqli/?id=1" -D dvwa --tables --batch
# dump a table
sqlmap -u "http://<kali-ip>/vulnerabilities/sqli/?id=1" -D dvwa -T users --dump --batch
```

Important: `sqlmap` is powerful; use only in your lab and explain it automates what you showed manually.

10. Show mitigation techniques (teach this after exploitation)

Parameterized queries / Prepared statements (strongest)
Example PHP PDO:

```
$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE user_id = ?');
$stmt->execute([$id]);
```

1. **Input validation & type enforcement**
   If `id` should be integer:
   `$id = intval($_GET['id']);`

2. **Use least-privilege DB accounts** — the web DB user should only have necessary rights (SELECT, maybe INSERT), not DROP/DELETE.

3. **Stored procedures / ORMs** — reduce raw SQL building.

4. **Web Application Firewall (WAF)** — adds a filtering layer (not a replacement for secure coding).

5. **Set DVWA security to Medium/High** after demo to demonstrate filtering and prepared statements.

11. Clean up / reset DVWA

After demo, reset DB and stop container:

```
# reset DB via DVWA UI: Setup → Create / Reset Database
# or remove containers
docker-compose down
# optionally remove images if desired
docker rmi vulnerables/web-dvwa:latest
```

Practical Activity:

1.Simulate a phishing email and identify red flags.

2. Perform a port scan with Nmap on a local machine.

1.Simulate a phishing email and identify red flags

Goal

Show how to craft a harmless training page + email, capture evidence when a student clicks, and identify red flags (social + technical).

1) Prepare a harmless training web page (teacher machine)

```
mkdir -p ~/phish_training && cd ~/phish_training

cat > index.html <<'HTML'

<!doctype html>

<html><body>

  <h1>Training Notice</h1>

  <p>This is a phishing training page. DO NOT enter credentials.</p>

</body></html>

HTML

# serve on port 8000 (bind to loopback for safety)

python3 -m http.server 8000 --bind 127.0.0.1 &

# note the PID if you need to kill: echo $!
```

- Accessible at `http://127.0.0.1:8000/` on teacher machine. If you want students to access it from a lab VM use the teacher VM IP and bind to 0.0.0.0.

2) Sample phishing email (display or send to test mailbox only)

Do NOT send externally. Put this in a text file or paste into GoPhish/test mail server.

```
From: helpdesk@school-support.com

To: student1@lab.local

Subject: IMPORTANT: Verify Your Student Portal Account

Hello Student,

We detected unusual activity on your student portal. Please verify your account within 24 hours to avoid
suspension.

Click: http://<TEACHER-IP>:8000/verify?user=student1

Regards,

IT Helpdesk

helpdesk@school-support.com
```

Red flags to ask students to find

- Urgency / threat: “within 24 hours”
- Generic greeting: “Hello Student” (no name)
- Mismatched sender domain vs official domain
- Suspicious URL (IP-based or short link) or link text != target
- Request to enter credentials / sensitive data
- Poor grammar/spelling, unusual contact numbers

3) (Optional) Send test email to lab mailbox using `swaks` (only if you have a lab SMTP)

```
# install swaks if needed

sudo apt update && sudo apt install -y swaks


# example (replace SMTP and addresses with lab values)

swaks --to student1@lab.local --from helpdesk@school-support.com \

  --server smtp.lab.local --header "Subject: IMPORTANT: Verify" \

  --body "This is a training message. Click http://<TEACHER-IP>:8000/verify"
```

If you don't have SMTP configured, just show the email content on screen.

4) Capture the click (tcpdump) while students safely open the training page

Start capture **on the teacher host** (loopback if local):

```
# loopback capture (if page served on localhost)

sudo tcpdump -i lo tcp port 8000 -w /tmp/phish_click_lo.pcap &

TCPDUMP_PID=$!
```

```
# or capture interface for external student clicks

sudo tcpdump -i eth0 tcp port 8000 -w /tmp/phish_click_eth0.pcap &

TCPDUMP_PID=$!
```

Have a volunteer open the link. Then stop capture:

```
sudo kill $TCPDUMP_PID
```

Open in Wireshark:

```
wireshark /tmp/phish_click_lo.pcap &
```

What to show in Wireshark

- HTTP GET request line (GET /verify?user=student1 HTTP/1.1)
- Host header (Host: <TEACHER-IP>:8000)
- User-Agent header (identify browser)
- Source IP of the click (student machine IP)
  → Explain how this metadata can show who clicked and what was requested.

5) Inspect the email headers & URL safely (commands)

Show students how to check technical signs:

- Extract domain from URL:

```
echo "http://<TEACHER-IP>:8000/verify?user=student1" | sed -E 's#https?://([^/]+).*#\1#'
```

- DNS lookup (if domain used):

```
dig +short suspicious-site.example.com
```

- Fetch headers (safe test):

```
curl -I http://127.0.0.1:8000/verify?user=student1
```

2. Perform a port scan with Nmap on a local machine (safe lab steps)

Goal

Teach students how to discover open ports/services on **localhost** or a lab VM, how to interpret results, and map ports to processes.

Run these on **your own machine** or lab VM only.

1) Basic localhost scan-Expected simplest output lists top ports and state (open/closed).

quick default scan of localhost

nmap 127.0.0.1

2) Faster, common ports scan (top 100)

sudo nmap -T4 --top-ports 100 127.0.0.1 -oN local_top100.txt

-oN saves human-readable output

3) SYN stealth scan + service/version detection (recommended)

sudo nmap -sS -sV -T4 127.0.0.1 -oA local_scan

-oA local_scan saves .nmap .xml .gnmap

Explain:

- `-sS` = SYN (stealth) scan (requires sudo)
- `-sV` = attempt to detect service & version
- `-T4` = faster timing (ok in lab)

4) Scan all TCP ports (very slow)

sudo nmap -p- 127.0.0.1 -oN all_ports_local.txt

5) Run vulnerability NSE scripts on lab VM only

sudo nmap -sV --script=vuln 192.168.56.101 -oN vuln_scan.txt

Warning: `--script=vuln` can be intrusive — use only on VMs you control.

6) Map ports to local processes (on the same machine)

After finding open ports, show which process is listening:

```
# Linux (Kali)

sudo ss -tulnp | grep LISTEN

# or

sudo lsof -i -P -n | grep LISTEN
```

This answers: *which program is serving on port 80, 22, 3306, etc.*

7) Interpret a simple nmap result (example)

PORT STATE SERVICE VERSION

22/tcp open ssh OpenSSH 8.4

80/tcp open http nginx 1.18

631/tcp open ipp CUPS 2.3

`PORT` → port number/protocol
`STATE` → open/closed/filtered
`SERVICE` → common name
`VERSION` → application/version (useful for CVE lookup)